



Object Oriented Programming

Pass Task 5.2: Case Study — Iteration 3: Bags

Overview

Object oriented programming makes best sense with larger programs. The case study will be your opportunity to create a larger program and better see how these abstractions make it easier to create software solutions.

- Purpose:** Progress on the case study, implementing bags to hold items.
- Task:** Understand the case study program and implement iteration 3.
- Time:** The end of Week 6, Fri 06, September 2024, 23:59:00 (Firmed)

Submission Details

All students have access to the Adobe Acrobat tools. Please print your solution to PDF and combine it with the screenshots taken for this task.

- Program source code
- Test source code
- Screenshot of unit tests passing

Instructions

1. Review the **Case Study Requirements** document. It outlines what you need to create.
2. For this week aim to complete Iteration 3.

Note: At this point there will not be a "program" as such, just a set of unit tests that help demonstrate that your solution is moving toward completion.

Once your tests are working correctly get a screenshot of the tests passing and submit them along with the code.

Assessment Criteria

Make sure that your task has the following in your submission:

- The program is implemented correctly based on the case study description.
- The “Universal Task Requirements” (see Canvas) have been met.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class Bag : Item
    {
        private Inventory _Inventory;

        public Bag(string[] ids, string name, string desc) : base(ids, name, desc)
        {
            _Inventory = new Inventory();
        }

        public GameObject Locate(string id)
        {
            if (AreYou(id))
            {
                return this;
            }
            return _Inventory.Fetch(id);
        }

        public override string FullDescription
        {
            get
            {
                return $"{this.Name}, containing:\n" + _Inventory.ItemList;
            }
        }

        }

        public Inventory Inventory
        {
            get
            {
                return _Inventory;
            }
        }
    }
}
```

```

    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class GameObject : IdentifiableObject
    {
        private string _name;
        private string _description;

        public GameObject(string[] ids, string name, string desc) : base(ids)
        {
            _name = name;
            _description = desc;
        }
        public string Name
        {
            get
            {
                return _name;
            }
        }
        public string ShortDescription
        {
            get
            {
                return $"{_name} ({FirstId})";
            }
        }
        public virtual string FullDescription
        {
            get
            {
                return _description;
            }
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Swin_Adventure2
{
    public class IdentifiableObject
    {
        private List<string> _Identifiers;

        public IdentifiableObject(string[] idents)
        {
            _Identifiers = new List<string>(idents);
        }
        public bool AreYou(string id)
        {
            return _Identifiers.Contains(id);
        }
        public string FirstId
        {
            get
            {
                return _Identifiers[0];
            }
        }
        public string AddIdentifier(string _Identifiers)
        {
            _Identifiers.ToLower();
            return _Identifiers;
        }
        public void PrivilegeEscalation(string pin)
        {
            if(pin.Length == 4)
            {

                pin = "1234"; //I don't know what a tutorial ID is
            }
        }
        public void ReplaceFirstId(string newId)
        {
            if (_Identifiers.Count > 0)
            {
                _Identifiers[0] = newId;
            }
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace Swin_Adventure2
{
    public class Inventory
    {
        private List<Item> _items;

        public Inventory()
        {
            _items = new List<Item>();
        }
        public bool HasItem(string id)
        {
            foreach (Item i in _items)
            {
                if (i.AreYou(id))
                {
                    return true;
                }
            }
            return false;
        }

        public void Put(Item itm)
        {
            _items.Add(itm);
        }
        public Item Take(string id)
        {
            foreach (Item item in _items)
            {
                if (item.AreYou(id))
                {
                    Item Found = item;
                    _items.Remove(item);
                    return Found;
                }
            }
            return null;
        }
        public Item Fetch(string id)
        {
            foreach (Item i in _items)
            {
                if (i.AreYou(id))
                {
                    return i;
                }
            }
        }
    }
}

```

```

        }return null;
    }
    public string ItemList
    {
        get
        {
            string listItem = "";
            foreach (Item item in _items)
            {
                listItem = listItem + item.ShortDescription + "\n";
            }
            return listItem;
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class Item : GameObject
    {
        public Item(string[] idents, string name, string desc) : base(idents, name, desc)
        {
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Swin_Adventure2
{
    public class Player : GameObject
    {
        private Inventory _Inventory;

        public Player(string name, string desc) : base(new string [] { "me", "inventory" }, name, desc)
        {
            _Inventory = new Inventory();
        }
    }
}

```

```

    }

    public GameObject Locate(string id)
    {
        if (AreYou(id))
        {
            return this;
        }
        return _Inventory.Fetch(id);
    }
    public override string FullDescription
    {
        get
        {
            return $"{Name}, you are carrying: \n" + _Inventory.ItemList;
        }
    }
}

public Inventory Inventory
{
    get
    {
        return _Inventory;
    }
}
}
}

```

```

namespace Swin_Adventure2
{
    internal class Program
    {
        static void Main(string[] args)
        {
            string studentId = "103995219";
            string firstName = "Callum";
            string lastName = "Fennessy";

            IdentifiableObject obj = new IdentifiableObject(new[] { "007", "James" });
            string tutorialId = "123";
            obj.ReplaceFirstId(tutorialId);
            string firstId = obj.FirstId;

            if (firstId == studentId)
            {
                Console.WriteLine("True");
            }
            else

```

```

        {
            Console.WriteLine("False");
        }
    }
}

using Swin_Adventure2;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using NUnit.Framework;

namespace Tests
{
    class BagTest
    {
        Bag bag;
        Bag bag1;
        Item shovel = new Item(new string[] { "shovel" }, "a shovel", "This is a shovel");
        Item sword = new Item(new string[] { "sword" }, "a sword", "This is a sword");
        Item pickaxe = new Item(new string[] { "pickaxe" }, "a pickaxe", "This is a pickaxe");
        Item axe = new Item(new string[] { "axe" }, "a axe", "This is a axe");
        Item hoe = new Item(new string[] { "hoe" }, "a hoe", "This is a hoe");
        Item bow = new Item(new string[] { "bow" }, "a bow", "This is a bow");

        [SetUp]
        public void Setup()
        {
            bag = new Bag(new string[] { "bag" }, "a bag", "This is a bag");
            bag1 = new Bag(new string[] { "bag1" }, "a bag1", "This is a bag1");
            bag.Inventory.Put(sword);
            bag.Inventory.Put(axe);
            bag.Inventory.Put(bow);
            bag1.Inventory.Put(shovel);
            bag1.Inventory.Put(hoe);
            bag1.Inventory.Put(pickaxe);
        }

        [Test]
        public void TestBagLocatesItems()
        {
            Assert.IsTrue(bag.Inventory.HasItem("sword"));
            Assert.IsTrue(bag.Inventory.HasItem("axe"));
            Assert.IsTrue(bag.Locate(sword.FirstId) == sword);
            Assert.IsTrue(bag.Locate(axe.FirstId) == axe);
        }

        [Test]

```

```

public void TestBagLocatesItself()
{
    Assert.IsTrue(bag.Locate(bag.FirstId) == bag);
    Assert.IsTrue(bag.Locate("bag") == bag);
}
[Test]
public void TestBagLocatesNothing()
{
    Assert.IsTrue(bag.Locate(hoe.FirstId) == null);
}
[Test]
public void TestBagFullDescription()
{
    Assert.AreEqual(bag.FullDescription, "a bag, containing:\na sword (sword)\na axe (axe)\na bow
(bow)\n");
}
[Test]
public void TestBagInBag()
{
    bag.Inventory.Put(bag1);
    Assert.IsTrue(bag.Locate(bag1.FirstId) == bag1);
    Assert.IsTrue(bag.Locate(sword.FirstId) == sword);
    Assert.IsTrue(bag.Locate(axe.FirstId) == axe);
}
}
}

```

